

Bloc 2 MVC PHP TD1

Contexte	2
Exercice 1	2
Exercice 2	3
Exercice 3	5
Exercice 4	6
Exercice 5	9
Exercice 6	11
Exercice 7	13
Exercice 8	16
Exercice 9	18
Conclusion	19

Contexte

Ce TD va consister en une série d'exercices qui se base principalement sur les bases du développement web en **PHP** et l'utilisation de **Git**, avec une mise en situation (les requêtes **HTTP** et les formulaires).

*(Les fichiers **.php** et **.html** seront disponibles dans le portfolio).*

Exercice 1

Création de la page "**page1.html**"

```
<!DOCTYPE html>
<html>
  <head>
    <title> Insérer le titrer ici </title>
  </head>

  <body>
    Un problème avec les accents à é è ?
    <!-- ceci est un commentaire -->
  </body>
</html>
```

file:///C:/xampp/htdocs/public_html/page1.html

Le "**file**" au début du lien veut dire que le fichier ne vient pas d'une page internet mais des fichiers locaux.

La page n'affiche pas les accents correctement, on utilise donc :

```
<meta charset="utf-8" />
```

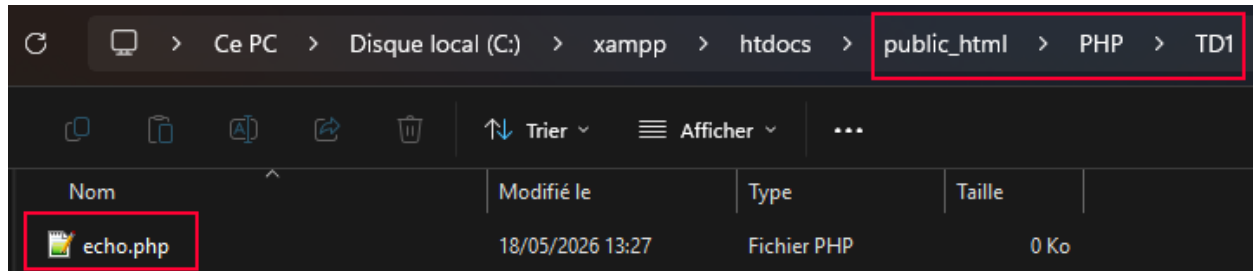
Il n'y a pas de communication client/serveur car le fichier viens de la machine

Sauf si on utilise **XAMPP**, étant donné que le fichiers **html** est dans les fichiers de XAMPP, on peut visualiser la page en **localhost** comme ceci :

"http://localhost/public_html/page1.html".

Exercice 2

Création du fichier “*echo.php*” dans “public_html\PHP\TD1”:



Avec le code suivant :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title> Mon premier php </title>
  </head>
  <body>
    Voici le résultat du script PHP :
    <?php
      // Ceci est un commentaire PHP sur une ligne
      /* Ceci est le 2ème type de commentaire PHP
      sur plusieurs lignes */

      // On met la chaîne de caractères "hello" dans la variable 'texte'
      // Les noms de variable commencent par $ en PHP
      $texte = "hello world !";

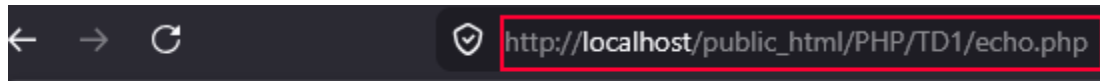
      // On écrit le contenu de la variable 'texte' dans la page Web
      echo $texte;
    ?>
  </body>
</html>
```

Si on l’ouvre comme t’elle, le navigateur ne sera capable que de lire l’HTML présent sur la page :

Voici le résultat du script PHP :

(Il n’y a rien d’autre d’afficher)

Cependant si on l'ouvre via le **localhost** (serveur), le **PHP** sera lu et appliqué :



Voici le résultat du script PHP : **hello world !**

- **Requête** : Le client **HTTP** (navigateur) demande un fichier **.php** au serveur.
- **Délégation** : Le serveur **HTTP** reconnaît l'extension et transmet la requête au moteur **PHP**.
- **Exécution** : Le moteur **PHP** lit et exécute le code sur le serveur (base de données, logique) pour générer un résultat (généralement du **HTML** ou du **JSON**).
- **Réponse** : Le serveur **HTTP** récupère ce résultat final et le renvoie au client.

Lorsqu'on fais **Ctrl + U**, on peut voir cette affichage :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title> Mon premier php </title>
  </head>

  <body>
    Voici le résultat du script PHP :
    hello world !    </body>
</html>
```

Exercice 3

On transforme le répertoire en dépôt **Git** puis on regarde son statut :

```
C:\xampp\htdocs\public_html\PHP>git init
Initialized empty Git repository in C:/xampp/htdocs/public_html/PHP/.git/

C:\xampp\htdocs\public_html\PHP>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    TD1/

nothing added to commit but untracked files present (use "git add" to track)
C:\xampp\htdocs\public_html\PHP>_
```

On exécute la commande “**git add TD1**” pour suivre les modifications de tous les fichiers dans le répertoire TD1, puis on reregarde son statut :

```
C:\xampp\htdocs\public_html\PHP>git add TD1

C:\xampp\htdocs\public_html\PHP>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   TD1/echo.php
```

Puis on **commit** :

```
C:\xampp\htdocs\public_html\PHP>git commit
hint: Waiting for your editor to close the file... _
```

Premier commit exo3

```
C:\xampp\htdocs\public_html\PHP>git commit
[master (root-commit) cd56378] Premier commit exo3
 1 file changed, 24 insertions(+)
 create mode 100644 TD1/echo.php
```

Exercice 4

Création de 3 variables :

```
$marque = "LaMarque";  
$couleur = "Rouge";  
$immatriculation = "XXX";
```

Ensuite, on affiche les variables :

```
echo "<p>Voiture $immatriculation de la marque $marque (couleur $couleur)</p>";
```

Voici le résultat du script PHP : hello world !

Voiture XXX de la marque LaMarque (couleur Rouge)

Ensuite, si on crée un tableau **\$voitures** :

```
$voitures = array ('marque' => 'LaMarque', 'couleur' => 'Rouge', 'immatriculation' => 'XXX');  
var_dump($voitures);
```

On peut ensuite utiliser **var_dump()** pour vérifier et extraire le contenu d'une variable :

```
array(3) { ["marque"]=> string(8) "LaMarque" ["couleur"]=> string(5) "Rouge" ["immatriculation"]=> string(3) "XXX" }
```

Si on souhaite faire le même affichage qu'au début, on doit faire ceci :

```
$voitures = array ('marque' => 'LaMarque', 'couleur' => 'Rouge', 'immatriculation' => 'XXX');  
echo "<p>Voiture " . $voitures['immatriculation'] . " de la marque " .  
$voitures['marque'] . " (couleur " . $voitures['couleur'] . ")</p>";
```

L'affichage est le même :

Voici le résultat du script PHP : hello world !

Voiture XXX de la marque LaMarque (couleur Rouge)

Voiture XXX de la marque LaMarque (couleur Rouge)

Ensuite, on crée une liste (un tableau indexé par des entiers) **\$voitures** de quelques “voitures-tableaux” et on vérifie le contenu avec **var_dump()** :

```
$voitures = array(
    array(
        'marque' => 'LaMarque',
        'couleur' => 'Rouge',
        'immatriculation' => 'XXX'
    ),
    array(
        'marque' => 'AutreMarque',
        'couleur' => 'Blanc',
        'immatriculation' => 'YYY'
    ),
    array(
        'marque' => 'AutreAutreMarque',
        'couleur' => 'Vert',
        'immatriculation' => 'ZZZ'
    )
);

var_dump($voitures);
```

La page donne ceci (**var_dump()**) :

```
Voici le résultat du script PHP : array(3) { [0]=> array(3) { ["marque"]=> string(8)
"LaMarque" ["couleur"]=> string(5) "Rouge" ["immatriculation"]=> string(3) "XXX" }
[1]=> array(3) { ["marque"]=> string(11) "AutreMarque" ["couleur"]=> string(5) "Blanc"
["immatriculation"]=> string(3) "YYY" } [2]=> array(3) { ["marque"]=> string(16)
"AutreAutreMarque" ["couleur"]=> string(4) "Vert" ["immatriculation"]=> string(3)
"ZZZ" } }
```

Affichage de la page “**finale**” avec listable de chaque voiture et une vérification si la liste est vide :

```
echo "<h2>Liste des voitures :</h2>";

if (empty($voitures)) {
    echo "<p>Il n'y a aucune voiture.</p>";
} else {
    echo "<ul>";

    foreach ($voitures as $voitureCourante) {
        echo "<li>Voiture " . $voitureCourante['immatriculation'] . " de la marque " .
            $voitureCourante['marque'] . " (couleur " . $voitureCourante['couleur'] . ")</li>";
    }
    echo "</ul>";
}

?>
```

Voici le résultat :

Liste des voitures :

- Voiture XXX de la marque LaMarque (couleur Rouge)
- Voiture YYY de la marque AutreMarque (couleur Blanc)
- Voiture ZZZ de la marque AutreAutreMarque (couleur Vert)

Si la liste est vide :

Liste des voitures :

Il n'y a aucune voiture.

Ensuite on fais le “git status”, cela dit qu’il y a un fichier qui a été modifié :

```
C:\xampp\htdocs\public_html\PHP>git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   TD1/echo.php

no changes added to commit (use "git add" and/or "git commit -a")
```

“git add” et “git commit” :

```
C:\xampp\htdocs\public_html\PHP>git add TD1/echo.php
C:\xampp\htdocs\public_html\PHP>git commit
hint: Waiting for your editor to close the file... _
```

TD1 Exo4 Affichage de variables

Résultat avec “git status” :

```
C:\xampp\htdocs\public_html\PHP>git commit
[master d332f21] TD1 Exo4 Affichage de variables
 1 file changed, 16 insertions(+), 12 deletions(-)
C:\xampp\htdocs\public_html\PHP>git status
On branch master
nothing to commit, working tree clean
C:\xampp\htdocs\public_html\PHP>
```


Exercice 5

Création de “Voitures.php” :

```
<?php
class Voiture {

    private $marque;
    private $couleur;
    private $immatriculation;

    // un getter
    public function getMarque() {
        return $this->marque;
    }

    // un setter
    public function setMarque($marque2) {
        $this->marque = $marque2;
    }

    // un constructeur
    public function __construct($m, $c, $i) {
        $this->marque = $m;
        $this->couleur = $c;
        $this->immatriculation = $i;
    }

    // une methode d'affichage.
    public function afficher() {
        // À compléter dans le prochain exercice
    }
}
?>
```

- Pour accéder à un attribut ou une fonction d'un objet, on utilise le “->” au lieu du “.”
- Le constructeur ne porte pas le nom de la classe, mais s'appelle **__construct()**, et il ne peut y avoir au maximum qu'un constructeur.

On fait ensuite les **Getter** et les **Setter** pour \$couleur et \$immatriculation :

```
public function getCouleur() {  
    return $this->couleur;  
}  
public function setCouleur($couleur2) {  
    $this->couleur = $couleur2;  
}  
  
public function getImmatriculation() {  
    return $this->immatriculation;  
}  
public function setImmatriculation($immatriculation2) {  
    $this->immatriculation = $immatriculation2;  
}
```

(*"2" car c'est une variable temporaire*).

On ajoute ensuite un check qui permet de ne pas rentrer d'immatriculation de plus de 8 caractères :

```
public function setImmatriculation($immatriculation2) {  
    if (strlen($immatriculation) <= 8) {  
        $this->immatriculation = $immatriculation;  
    } else {  
        echo "Erreur : L'immatriculation saisie est trop longue (8 caractères maximum).";  
    }  
}
```

On doit ensuite préparé la fonction d'affichage, comme ceci :

```
public function afficher() {  
    echo "<p>Voiture " . $this->immatriculation . " de la marque " . $this->marque  
    . " (couleur " . $this->couleur . "</p>";  
}
```

Pour pouvoir vérifier la création d'objet et l'affichage, on doit faire ceci :

```
$maVoiture = new Voiture("MarqueQuelquonque", "Rose", "AZERTYUI");  
$maVoiture->afficher();
```

L'affichage de la page fonctionne :

Voiture AZERTYUI de la marque MarqueQuelquonque (couleur Rose)

Puis on le commit sur le **Git** :

```
C:\Windows\System32>cd C:\xampp\htdocs\public_html\PHP

C:\xampp\htdocs\public_html\PHP>git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        TD1/Voitures.php

nothing added to commit but untracked files present (use "git add" to track)

C:\xampp\htdocs\public_html\PHP>git add TD1/Voitures.php

C:\xampp\htdocs\public_html\PHP>git commit
[master 97c3fae] Add Voitures.php
 1 file changed, 54 insertions(+)
 create mode 100644 TD1/Voitures.php

C:\xampp\htdocs\public_html\PHP>git status
On branch master
nothing to commit, working tree clean

C:\xampp\htdocs\public_html\PHP>
```

Exercice 6

Création du fichier “**testVoiture.php**” :



Ce fichier doit avoir le squelette classique **HTML** (*footer* n'est pas nécessaire ici) :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title> testVoiture </title>
  </head>

  <body>
    <?php
      ?>
    </body>
</html>
```

Puis on récupère le fichier “**Voiture.php**” (étant donné que le fichier est dans le même dossier que le fichier actuel, on a pas besoin d’écrire le chemin “\...\” :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title> testVoiture </title>
  </head>
  <body>
    <?php
    require_once('Voiture.php');
    ?>
  </body>
</html>
```

Initialisation d’une variable “***\$voiture1***” de la classe **Voiture** :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title> testVoiture </title>
  </head>
  <body>
    <?php
    require_once('Voiture.php');
    $voiture1 = new Voiture("PetitPoney", "Rose Pastel", "AZERTYUI");
    ?>
  </body>
</html>
```

Puis on l’affiche avec la fonction “**afficher()**” :

```
$voiture1->afficher();
```

Résultat :

Voiture AZERTYUI de la marque PetitPoney (couleur Rose Pastel)

Puis on fais un commit **Git** :

```
C:\xampp\htdocs\public_html\PHP>git add TD1/Voiture.php TD1/testVoiture.php

C:\xampp\htdocs\public_html\PHP>git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   TD1/Voiture.php
    new file:   TD1/testVoiture.php

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    TD1/Voitures.php

C:\xampp\htdocs\public_html\PHP>git commit
[master c02579e] Add testVoiture.php and rename Voitures.php to Voiture.php (no s)
 2 files changed, 69 insertions(+)
 create mode 100644 TD1/Voiture.php
 create mode 100644 TD1/testVoiture.php

C:\xampp\htdocs\public_html\PHP>git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    TD1/Voitures.php
```

Exercice 7

Création de “**formulaireVoiture.html**” avec la même entête que “**echo.php**” :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title> Voitures </title>
  </head>

  <body>
    <?php
      <form method="get" action="creerVoiture.php">
        <fieldset>
          <legend>Mon formulaire :</legend>
          <p>
            <label for="immat_id">Immatriculation</label> :
            <input type="text" placeholder="Ex : 256AB34" name="immatriculation" id="immat_id" required/>
          </p>
          <p>
            <input type="submit" value="Envoyer" />
          </p>
        </fieldset>
      </form>
    <?>
  </body>
</html>
```

Ajout des champs “marque” et “couleur” dans le formulaire :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title> Voitures </title>
  </head>
  <body>
    <form method="get" action="creerVoiture.php">
      <fieldset>
        <legend>Mon formulaire :</legend>
        <p>
          <label for="marque_id">Marque</label> :
          <input type="text" placeholder="Ex : Opel" name="marque" id="marque_id" required/>
        </p>
        <p>
          <label for="couleur_id">Couleur</label> :
          <input type="text" placeholder="Ex : Rouge" name="couleur" id="couleur_id" required/>
        </p>
        <p>
          <label for="immat_id">Immatriculation</label> :
          <input type="text" placeholder="Ex : 256AB34" name="immatriculation" id="immat_id" required/>
        </p>
        <p>
          <input type="submit" value="Envoyer" />
        </p>
      </fieldset>
    </form>
  </body>
</html>
```

Résultat :

Mon formulaire :

Marque :

Couleur :

Immatriculation :

En cliquant sur “**Envoyer**” cela va rediriger vers “**creerVoiture.php**” avec ce qu’on a mis dans le formulaire en paramètres (grâce à `method="get"`), sauf que la page n’existe pas encore :



La partie après la partie rouge s’appelle une “**chaîne de requête**”.
(`?marque=ff&couleur=fff&immatriculation=fff`)

Puis le fichier **.html** a été commit sur **Git** :

On peut voir les logs du **Git** avec “**log git**” :

```
C:\xampp\htdocs\public_html\PHP>git log
commit 1731f83070ddeeb20ab4d13dad5d5e3b60fa080 (HEAD -> master)
Author: RA <aubree.rom@gmail.com>
Date: Tue May 19 13:23:21 2026 +0200

    update Voiture.php

commit ea044a220d11672a799d1ea8c37242ad123077f9
Author: RA <aubree.rom@gmail.com>
Date: Tue May 19 13:22:46 2026 +0200

    Ajout de formulaireVoiture

commit c02579efe28f8a688d3e017acf107daf7ccee6cb
Author: RA <aubree.rom@gmail.com>
Date: Tue May 19 10:46:59 2026 +0200

    Add testVoiture.php and rename Voitures.php to Voiture.php (no s)

commit 97c3faebf02f58f5be70af76b31095e56f16f919
Author: RA <aubree.rom@gmail.com>
Date: Tue May 19 10:26:31 2026 +0200

    Add Voitures.php

commit d332f212edf6af37588b39e11d06217694bd0025
Author: RA <aubree.rom@gmail.com>
Date: Mon May 18 15:14:04 2026 +0200

    TD1 Exo4 Affichage de variables
:
commit 1731f83070ddeeb20ab4d13dad5d5e3b60fa080 (HEAD -> master)
Author: RA <aubree.rom@gmail.com>
Date: Tue May 19 13:23:21 2026 +0200

    update Voiture.php
```

Exercice 8

Création de “**creerVoiture.php**”, On doit récupérer les informations du formulaire avec “**\$_GET**” et on vérifie si elle sont initialisés avec “**isset**”, on utilise aussi “**htmlspecialchars**” qui permet de prendre une chaîne de caractères spéciaux et de la convertir en entités **HTML** :

```
if (isset($_GET['marque']) && isset($_GET['couleur']) && isset($_GET['immatriculation'])) {  
  
    $marque = $_GET['marque'];  
    $couleur = $_GET['couleur'];  
    $immatriculation = $_GET['immatriculation'];  
  
    echo "<li>" . htmlspecialchars($marque) . "</li>";  
    echo "<li>" . htmlspecialchars($couleur) . "</li>";  
    echo "<li>" . htmlspecialchars($immatriculation) . "</li>";  
}
```

Le formulaire :

Mon formulaire :

Marque :

Couleur :

Immatriculation :

Résultat (affichage simple) :

MarqueTest
CouleurTest
ImmatTest

On modifie le code de **formulaireVoiture.html** pour qu'elle utilise **"post"** plutôt que **"get"**, comme ça, les informations ne seront pas visible dans le lien :

"formulaireVoiture.html" :

```
</head>
<body>
  <form method="post" action="creerVoiture.php">
    <fieldset>
      <legend>Mon formulaire :</legend>
      <p>
```

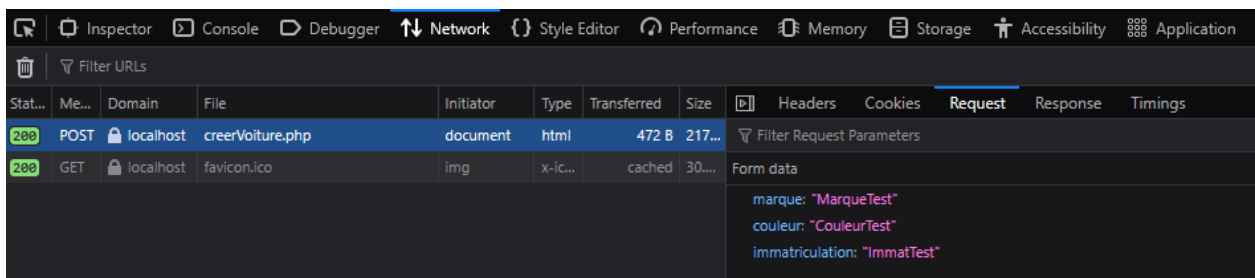
"creerVoiture.php" :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title> testVoiture </title>
  </head>
  <body>
    <?php
    if (isset($_POST['marque']) && isset($_POST['couleur']) && isset($_POST['immatriculation'])) {
      $marque = $_POST['marque'];
      $couleur = $_POST['couleur'];
      $immatriculation = $_POST['immatriculation'];

      echo "<li>" . htmlspecialchars($marque) . "</li>";
      echo "<li>" . htmlspecialchars($couleur) . "</li>";
      echo "<li>" . htmlspecialchars($immatriculation) . "</li>";
      require_once('Voiture.php');

    }
    ?>
  </body>
</html>
```

Voici la requête sur la page **"creerVoiture.php"** :



The screenshot shows the Network tab of a web browser's developer tools. A POST request to `localhost/creerVoiture.php` is selected. The request details show the following form data:

Stat...	Me...	Domain	File	Initiator	Type	Transferred	Size	Headers	Cookies	Request	Response	Timings	
200	POST	localhost	creerVoiture.php	document	html	472 B	217...	Filter Request Parameters					
200	GET	localhost	favicon.ico	img	x-ic...	cached	30...	Form data					

The form data for the POST request is:

- marque: "MarqueTest"
- couleur: "CouleurTest"
- immatriculation: "ImmatTest"

Puis, on **commit** sur **Git** :

```
C:\xampp\htdocs\public_html\PHP>git add TD1/creerVoiture.php TD1/formulaireVoiture.html

C:\xampp\htdocs\public_html\PHP>git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   TD1/creerVoiture.php
    modified:   TD1/formulaireVoiture.html

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    TD1/Voitures.php

C:\xampp\htdocs\public_html\PHP>git commit
[master 899afd2] create creerVoiture.php and change "get" to "post" :
 2 files changed, 25 insertions(+), 1 deletion(-)
 create mode 100644 TD1/creerVoiture.php

C:\xampp\htdocs\public_html\PHP>git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    TD1/Voitures.php

no changes added to commit (use "git add" and/or "git commit -a")
```

Exercice 9

Création des classes “**Utilisateur**” et “**Trajet**” pour les bases d’un site de covoiturage.

Pour éviter de taper 7 getters, 7 setters et un constructeur avec 7 arguments pour **Trajet**, nous allons faire :

- Des **getters** génériques **get(\$nom_attribut)** qui renvoient l’attribut de nom **\$nom_attribut**. Utilisez la syntaxe suivante pour accéder à l’attribut de nom **\$nom_attribut** de l’objet **\$objet** : “**\$objet->\$nom_attribut**”
- Des **setters** génériques **set(\$nom_attribut, \$valeur)** ;
- Un constructeur **__construct(\$data)** qui prend un tableau dont les index correspondent aux attributs de la classe.

Utilisateur :

```
<?php
class Utilisateur {
    private $login;
    private $nom;
    private $prenom;

    public function __construct($data) {
        foreach ($data as $cle => $valeur) {
            $this->$cle = $valeur;
        }
    }

    public function get($nom_attribut) {
        return $this->$nom_attribut;
    }

    public function set($nom_attribut, $valeur) {
        $this->$nom_attribut = $valeur;
    }
}
?>
```

Trajet :

```
<?php
class Trajet {
    private $id;
    private $depart;
    private $arrivee;
    private $date;
    private $nbplaces;
    private $prix;
    private $conducteur_login;

    public function __construct($data) {
        foreach ($data as $cle => $valeur) {
            $this->$cle = $valeur;
        }
    }

    public function get($nom_attribut) {
        return $this->$nom_attribut;
    }

    public function set($nom_attribut, $valeur) {
        $this->$nom_attribut = $valeur;
    }
}
?>
```

Conclusion

Ce **TD** nous a introduit à l'architecture client-serveur et à la programmation orientée objet en **PHP**, et à sa mise en pratique via différents exercices (les classes Voiture, Utilisateur et Trajet).